

Plateforme de Soutien Scolaire , Documentation Technique de l'API REST

Ce composant applicatif constitue l'API back-end développée avec le framework Spring Boot pour orchestrer une plateforme de soutien scolaire. Elle gère la mise en relation entre élèves, enseignants et administrateurs à travers un référentiel de matières, un suivi du cycle de vie des demandes d'accompagnement et un module de messagerie contextuelle bidirectionnelle. La sécurité est opérée de manière stateless par des jetons JWT avec une granularité de droits basée sur les rôles et la propriété des données.

Protocole de Recette Rapide de l'API

L'application s'expose par défaut sur l'URL `http://localhost:8080` . Afin de procéder à la validation fonctionnelle, veuillez à initialiser préalablement l'infrastructure à l'aide de la commande suivante :

```
docker compose up --build
```

Méthode 1 : Import de Collection Postman

1. Accédez à l'interface de Postman et utilisez l'option d'importation pour charger les deux fichiers localisés dans le répertoire `postman/` : * `Plateforme-Soutien-Scolaire.postman_collection.json` * `Plateforme-Soutien-Scolaire.postman_environment.json`
2. Sélectionnez l'environnement nommé « Plateforme Soutien Scolaire - Local ».
3. Exécutez les requêtes de manière séquentielle en suivant l'ordre des répertoires (dossiers 1 à 8), ou initiez le module Collection Runner.

Note : Les scripts de la collection capturent de façon dynamique les jetons JWT d'authentification ainsi que les variables d'entités (identifiants de matières et de requêtes) pour éliminer tout besoin de manipulation manuelle.

Méthode 2 : Interface Swagger UI

1. Naviguez vers l'URL : `http://localhost:8080/swagger-ui.html` .
2. Soumettez une requête sur le point d'accès `POST /api/auth/register` ou `POST /api/auth/login` afin d'extraire la valeur du jeton de sécurité (`token`).
3. Utilisez le bouton **Authorize** en haut à droite de l'interface, collez la chaîne du jeton (sans le préfixe) et validez. Pour modifier le contexte d'authentification ou changer de rôle, réitérez l'opération avec un autre jeton.

Méthode 3 : Client REST intégré (requests.http)

Ouvrez le fichier `requests.http` situé à la racine du projet depuis un environnement de développement compatible (extension REST Client pour VS Code ou utilitaire HTTP d'IntelliJ). Activez les requêtes séquentiellement via les boutons d'exécution intégrés.

1. Stack Technique Appliquée

Composant	Technologie et Version
Langage	Java 17
Framework Global	Spring Boot 3.3.5 (Modules: Web, Data JPA, Security, Validation)
Persistance (Production)	PostgreSQL 16 (Déploiement conteneurisé)
Persistance (Tests)	Base de données H2 (Exécution en mémoire)
Sécurité Applicative	Spring Security + JWT (Bibliothèque jjwt)
Documentation API	OpenAPI 3 / Swagger UI (Via la dépendance springdoc)
Gestionnaire de Build	Apache Maven

2. Prérequis Système

- Java Development Kit (JDK) 17 ou version supérieure.
- Apache Maven 3.9 ou version supérieure.
- Docker Engine et Docker Compose opérationnels.

3. Installation et Déploiement

Étape 0 : Configuration des Variables d'Environnement

Les données de configuration critiques (identifiants de base de données et clé privée de signature JWT) sont externalisées du code source. Initialisez le fichier d'environnement à partir du modèle d'exemple :

```
cp .env.example .env
```

(Pour les environnements Windows, utilisez la commande : `copy .env.example .env`)

Option A : Déploiement Conteneurisé Intégral

Cette méthode compile le code source et instancie simultanément le serveur d'application et le serveur de données au sein d'un réseau isolé :

```
docker compose up --build
```

L'API devient accessible sur `http://localhost:8080` dès que le script de vérification d'état interne (*healthcheck*) valide la disponibilité opérationnelle de PostgreSQL. Pour une exécution en arrière-plan, ajoutez l'argument `-d`.

Option B : Déploiement Hybride (Développement)

1. Instanciez uniquement l'unité de stockage PostgreSQL conteneurisée :

```
docker compose up -d postgres
```

2. Initiez le serveur d'application Spring Boot sur le runtime local :

```
mvn spring-boot:run
```

Commandes d'Arrêt de l'Infrastructure

- Arrêt simple des conteneurs avec préservation des volumes de données : `docker compose down`.
 - Arrêt complet avec destruction des volumes et réinitialisation de la base : `docker compose down -v`.
-

4. Matrice des Rôles et Droits d'Accès

Fonctionnalité Métier	STUDENT	TEACHER	ADMIN
Inscription et authentification	Autorisé	Autorisé	Autorisé
Consultation du catalogue des matières	Autorisé	Autorisé	Autorisé
Création / Modification / Suppression de matières	Non autorisé	Non autorisé	Autorisé
Publication d'une demande de soutien scolaire	Autorisé	Non autorisé	Non autorisé
Consultation des demandes en attente (CREATED)	Non autorisé	Autorisé	Autorisé
Affectation et prise en charge d'une demande	Non autorisé	Autorisé	Non autorisé
Clôture définitive d'une demande (COMPLETED)	Autorisé (Propriétaire)	Non autorisé	Autorisé
Annulation d'une demande de soutien (CANCELLED)	Autorisé (Propriétaire)	Non autorisé	Autorisé
Lecture et écriture de messages contextuels	Autorisé (Participant)	Autorisé (Participant)	Autorisé (Lecture seule)
Extraction de la liste et détails des utilisateurs	Non autorisé	Non autorisé	Autorisé

5. Référentiel des Points d'Accès REST

Authentification (Accès Public)

- `POST /api/auth/register` , Enregistrement d'un nouvel utilisateur (Retourne le jeton JWT).
- `POST /api/auth/login` , Validation des identifiants et délivrance du jeton JWT.

Gestion des Matières

- `GET /api/subjects` , Extraction du catalogue des matières (Tout utilisateur authentifié).
- `POST /api/subjects` , Insertion d'une nouvelle matière (ADMIN).
- `PUT /api/subjects/{id}` , Mise à jour complète d'une matière existante (ADMIN).
- `DELETE /api/subjects/{id}` , Suppression d'une matière du référentiel (ADMIN).

Gestion des Demandes de Soutien

- `POST /api/requests` , Soumission d'une nouvelle demande d'accompagnement (STUDENT).

- `GET /api/requests/available` , Liste des demandes en attente de prise en charge (TEACHER, ADMIN).
- `GET /api/requests/mine` , Récupération des demandes propres à l'utilisateur connecté (Filtre dynamique selon le rôle).
- `GET /api/requests/{id}` , Consultation détaillée des paramètres d'une demande (Participants directs, ADMIN).
- `POST /api/requests/{id}/assign` , Liaison d'un enseignant à une demande disponible (TEACHER).
- `PATCH /api/requests/{id}/complete` , Transition d'une demande active vers le statut terminé (STUDENT créateur, ADMIN).
- `PATCH /api/requests/{id}/cancel` , Transition d'une demande vers le statut annulé (STUDENT créateur, ADMIN).

Module de Messagerie Contextuelle

- `POST /api/requests/{id}/messages` , Transmission d'un message lié à une demande active (Participants directs).
- `GET /api/requests/{id}/messages` , Récupération chronologique de l'historique des échanges (Participants directs, ADMIN).

Administration des Utilisateurs

- `GET /api/users` , Liste exhaustive des comptes enregistrés sur la plateforme (ADMIN).
- `GET /api/users/{id}` , Consultation des informations détaillées d'un compte spécifique (ADMIN).

6. Exemples de Commandes cURL (Syntaxe Bash)

6.1 Processus d'Inscription

```
curl -X POST http://localhost:8080/api/auth/register \
-H "Content-Type: application/json" \
-d '{"fullName":"Eleve","email":"eleve@test.com","password":"eleve123","role":"STUDENT"}'

curl -X POST http://localhost:8080/api/auth/register \
-H "Content-Type: application/json" \
-d '{"fullName":"Prof","email":"prof@test.com","password":"prof123","role":"TEACHER"}'

curl -X POST http://localhost:8080/api/auth/register \
-H "Content-Type: application/json" \
-d '{"fullName":"Admin","email":"admin@test.com","password":"admin123","role":"ADMIN"}'
```

6.2 Processus d'Authentification

```
curl -X POST http://localhost:8080/api/auth/login \  
-H "Content-Type: application/json" \  
-d '{"email":"admin@test.com","password":"admin123"}'
```

6.3 Création d'une Matière (Requiert les privilèges Administrateur)

```
curl -X POST http://localhost:8080/api/subjects \  
-H "Authorization: Bearer $TOKEN_ADMIN" \  
-H "Content-Type: application/json" \  
-d '{"name":"Reinforcement Learning ","description":"env : CarPol"}'
```

6.4 Soumission d'une Demande de Soutien (Requiert les privilèges Élève)

```
curl -X POST http://localhost:8080/api/requests \  
-H "Authorization: Bearer $TOKEN_ELEVE" \  
-H "Content-Type: application/json" \  
-d '{"subjectId":1,"description":"Maintenir la tige droite, éviter la descente"}'
```

6.5 Consultation et Affectation par un Enseignant

```
# Consultation des demandes en attente  
curl http://localhost:8080/api/requests/available \  
-H "Authorization: Bearer $TOKEN_PROF"  
  
# Prise en charge de la demande spécifiée par l'ID 1  
curl -X POST http://localhost:8080/api/requests/1/assign \  
-H "Authorization: Bearer $TOKEN_PROF"
```

6.6 Échanges au sein de la Messagerie Contextuelle

```
# Publication d'un message par l'élève
curl -X POST http://localhost:8080/api/requests/1/messages \
  -H "Authorization: Bearer $TOKEN_ELEVE" \
  -H "Content-Type: application/json" \
  -d '{"content":"Bonjour, je bloque sur cet exercice"}'

# Réponse émise par l'enseignant affecté
curl -X POST http://localhost:8080/api/requests/1/messages \
  -H "Authorization: Bearer $TOKEN_PROF" \
  -H "Content-Type: application/json" \
  -d '{"content":"Bonjour, envoie-moi un exemple précis"}'

# Extraction de l'historique complet des discussions
curl http://localhost:8080/api/requests/1/messages \
  -H "Authorization: Bearer $TOKEN_ELEVE"
```

6.7 Clôture de la Demande par l'Élève Propriétaire

```
curl -X PATCH http://localhost:8080/api/requests/1/complete \
  -H "Authorization: Bearer $TOKEN_ELEVE"
```

7. Exécution de la Suite de Tests

L'ensemble des tests automatisés (unitaires et d'intégration) s'exécute de manière isolée via la commande suivante :

```
mvn test
```

Note : Le profil de test est configuré pour basculer automatiquement sur une base de données H2 intégrée en mémoire, éliminant ainsi la dépendance à une instance externe ou à un conteneur PostgreSQL actif.

8. Organisation Structurale du Projet

```
src/main/java/com/soutien
```

```
|— PlateformeSoutienApplication.java  # Point d'entrée de l'application Spring Boot
|— config/                          # Configurations de sécurité et de documentation OpenAPI
|— controller/                      # Couche d'exposition des points d'accès REST (HTTP)
|— service/                         # Couche d'implémentation de la logique métier et des tra
|— repository/                     # Couche d'accès aux données (Interfaces Spring Data JPA)
|— entity/                         # Modèles du domaine persistant et énumérations associées
|— dto/                            # Objets d'échange de données (Modèles de requêtes et répo
|— exception/                      # Définition des exceptions customisées et gestionnaire d
|— security/                       # Composants d'infrastructure liés à la sécurité JWT
```